

OOP in .NET — Questions 20 to 31

Interview Questions & Correct Answers

این فایل شامل سؤال‌های ۲۰ تا ۳۱ از تمرین همین جلسه و جواب‌های اصلاح‌شده و مناسب مصاحبه است.

20. Association vs Aggregation vs Composition

Question: What is the difference between Association, Aggregation, and Composition in OOP?

Interview answer: Association means two objects are related. Aggregation is a weak whole-part relationship where the child can exist independently of the parent. Composition is a strong whole-part relationship where the child usually belongs to the parent and does not have a meaningful independent lifecycle.

باشند، Aggregation می‌توانند Employee و Department باشند، Association می‌توانند Order و Customer: نکته: مثال ساده هستند Composition معمولاً OrderItem و Order و

21. Cohesion

Question: What is cohesion in OOP, and why is high cohesion good?

Interview answer: Cohesion means how closely related the responsibilities and members of a class are. High cohesion means a class focuses on a clear responsibility, which makes the code easier to understand, test, and maintain.

آن بالاتر است cohesion، نکته: هرچه اعضای یک کلاس حول یک مسئولیت مشخص‌تر باشند

22. Coupling

Question: What is coupling in OOP, and why do we usually prefer low coupling?

Interview answer: Coupling means how dependent classes are on each other. Low coupling is preferred because changes in one class have less impact on other classes, which improves maintainability, testability, and flexibility.

نکته: وابستگی کمتر بین کلاس‌ها یعنی تغییرات محلی‌تر و تست ساده‌تر

23. High Cohesion + Low Coupling

Question: Why do we usually want high cohesion and low coupling in software design?

Interview answer: High cohesion means a class focuses on one clear responsibility. Low coupling means changes in one class have less impact on other classes. Together, they make a system easier to understand, test, maintain, and change.

نکته: هدف طراحی خوب این است که هر کلاس کار خودش را متمرکز انجام دهد و کمترین وابستگی لازم را به بقیه داشته باشد

24. Reference Equality vs Value Equality

Question: What is the difference between reference equality and value equality in C#?

Interview answer: Reference equality checks whether two references point to the same object instance. Value equality checks whether two objects are logically equal based on their values, even if they are different instances.

```
var a = new Person { Name = "Hamed" };  
var b = a;
```

```
ReferenceEquals(a, b); // true
```

```
var x = new Person { Name = "Hamed" };  
var y = new Person { Name = "Hamed" };  
ReferenceEquals(x, y); // false
```

assignment. درباره مقایسه است، نه صرفاً equality یکی نیست؛ equality با reference نکته: کیپی

25. == vs Equals()

Question: What is the difference between == and Equals() in C#?

Interview answer: `==` is an operator and its behavior depends on the type and whether the operator is overloaded. `Equals()` is a method that can be overridden to define logical value equality. For many ordinary reference types, the default behavior is reference equality, while types such as string provide value-based equality.

؛ رفتار به نوع و پیاده‌سازی value equality یعنی Equals و reference equality نمی‌توان گفت همیشه == یعنی C# نکته: در بستگی دارد.

26. GetHashCode with Equals

Question: Why should we override GetHashCode() when we override Equals()?

Interview answer: Because equal objects must return the same hash code. Hash-based collections such as Dictionary and HashSet use GetHashCode() to locate a bucket and then use equality checks, so GetHashCode() must be consistent with Equals().

آن دو باید یکسان باشد hash code، باشد true برابر a.Equals(b) نکته: قانون مهم: اگر

27. Value Type vs Reference Type

Question: What is the difference between value types and reference types in C#?

Interview answer: A value type stores its value directly, so copying it copies the value. A reference type variable stores a reference to an object, so copying the variable copies the reference and both variables can refer to the same object. Memory placement depends on context; the simple rule value type = stack and reference type = heap is not always correct.

را مطلق بیان نکن stack/heap است؛ بحث semantics نکته: مهم‌ترین تفاوت مصاحبه‌ای رفتار کیپی و

28. class vs struct

Question: What is the difference between a class and a struct in C#?

Interview answer: A class is a reference type, while a struct is a value type. Copying a class variable copies the reference; copying a struct copies its value. Classes support class inheritance, while structs cannot inherit from another class or struct.

دیگری ارث‌بری نمی‌کند struct یا class پیاده‌سازی کند، ولی از interface می‌تواند struct نکته:

29. When to Use struct

Question: When would you choose a struct instead of a class in C#?

Interview answer: Use a struct for small, value-like data that does not need object identity or class inheritance. Structs are often a good fit when value semantics are more natural, and they are commonly designed to be immutable.

```
public readonly struct Point
{
    public int X { get; }
    public int Y { get; }

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

مهمتر است identity بودن و نداشتن value-like نکته: کوچک بودن داده به‌تنهایی کافی نیست؛

30. record vs class

Question: What is the difference between a record and a class in C#?

Interview answer: A record is useful for data-centric types where value-based equality is important. Records are commonly used with immutable-style designs, but they are not required to be immutable. A normal class uses reference-based equality by default unless equality is customized.

```
public record Person(string Name, int Age);

var p1 = new Person("Hamed", 40);
var p2 = new Person("Hamed", 40);
Console.WriteLine(p1 == p2); // true
```

مهمتر است data-centric modeling و value equality بودن نیست؛ immutable فقط record نکته: هدف اصلی

31. record class vs record struct

Question: What is the difference between record class and record struct in C#?

Interview answer: `record class` is a reference type, while `record struct` is a value type. Both provide generated value-based equality, but their copy semantics differ: a record class variable copies a reference, while a record struct copies its value.

```
public record Person(string Name);
public record struct Point(int X, int Y);
```

است value type و دیگری reference type مناسب تولید می‌کنند، اما یکی equality هستند و record نکته: هر دو

Quick Review

Concept	Short definition
Association	General relationship between objects
Aggregation	Weak whole-part; child can live independently
Composition	Strong whole-part; child lifecycle tied to parent
Cohesion	How related the responsibilities inside a class are
Coupling	How dependent classes are on each other
Reference equality	Same object instance
Value equality	Logically equal values
GetHashCode	Must be consistent with Equals
Value type	Copy the value
Reference type	Copy the reference
struct	Value type; value-like data
record	Data-centric type with generated value equality